VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018

MLOps Mini Project Report on

"E-COMMERCE SALES ANALYTICS AND CUSTOMER-CHURN MLOps PIPELINE"

Submitted in Partial fulfilment of the Requirements for the Degree of
Bachelor of Engineering in Computer Science and Engineering (Artificial Intelligence & Machine Learning)

By

RACHIT JOSEPH (1CR23CI041)

Under the Guidance of,

Dr Swastisudha Punyatoya
Dept. of AI & ML

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

CMR INSTITUTE OF TECHNOLOGY

Affiliated to VTU, Approved by AICTE, Accredited by NBA and NAAC with "A++" Grade
ITPL MAIN ROAD, BROOKFIELD, BENGALURU-560037, KARNATAKA, INDIA

CMR INSTITUTE OF TECHNOLOGY

Affiliated to VTU, Approved by AICTE, Accredited by NBA and NAAC with "A++" Grade
ITPL MAIN ROAD, BROOKFIELD, BENGALURU-560037, KARNATAKA, INDIA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

CERTIFICATE

This is to certify that the Mini Project report work entitled **"E-Commerce Sales Analytics and Customer-Churn MLOps Pipeline"** has been carried out by **Rachit Joseph (1CR23CI041)**, a Bonafide student of CMR Institute of Technology, Bengaluru in partial fulfilment for the award of the Degree of **Bachelor of Engineering in Computer Science and Engineering (Artificial Intelligence and Machine Learning)** of the Visvesvaraya Technological University, Belagavi during the year 2026-2027. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the departmental library. This Mini Project report has been approved as it satisfies the academic requirements in respect of Mini Project work prescribed for the said Degree.

Signature of
Dr Swastisudha Punyatoya
Project Guide
Dept. of AI & ML
CMR Institute of Technology

Signature of
Dr. Shyam P Joy
Head of Department
Dept. of AI & ML
CMR Institute of Technology

DECLARATION

Mr. Rachit Joseph (1CR23CI041) hereby declare that the Mini Project report entitled **"E-Commerce Sales Analytics and Customer-Churn MLOps Pipeline"** has been carried out by me under the guidance of **Dr Swastisudha Punyatoya**, Department of AI & ML, CMR Institute of Technology, Bengaluru, in partial fulfilment of the requirement for the degree of **BACHELOR OF ENGINEERING IN COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)**, of Visvesvaraya Technological University, Belagavi during the academic year 2026-2027. The work done in this Mini Project report is original and it has not been submitted for any other degree in any university.

Place: Bangalore

Date: 31/10/2026

Rachit Joseph (1CR23CI041)

ABSTRACT

Retail businesses accumulate large volumes of transactional data, but that data only becomes useful once it is consolidated, validated and modelled. This project implements a complete, production-shaped data and machine-learning platform over the UCI Online Retail II dataset: an orchestrated ETL pipeline that loads 1,067,371 raw transaction rows through validation into a PostgreSQL star schema, an analytics dashboard over the resulting warehouse, and a supervised customer-churn model served through a REST API.

The pipeline is orchestrated by Apache Airflow. Monthly landing files are ingested incrementally into a raw layer, validated against explicit business rules into a staging layer, and transformed into a dimensional warehouse with fact and dimension tables plus aggregate marts. Of the 1,067,371 raw rows, 768,873 passed validation and 261,822 were rejected with recorded reasons, and 768,865 fact rows were loaded. Every task is idempotent, so re-running the pipeline never duplicates data.

The machine-learning component predicts customer churn, defined as placing no order in the 90 days following a month-end snapshot. Features are computed strictly from data on or before each cutoff to prevent target leakage, producing 62,607 labelled customer-month snapshots across 16 cutoffs. Three candidate models — logistic regression, random forest and gradient-boosted trees — were trained and compared on a chronological train/validation/test split, so that no customer's future can inform their own prediction. The random forest was selected on validation ROC-AUC and achieved ROC-AUC 0.782, precision 0.677 and recall 0.952 on a strictly later held-out test block.

Beyond training, the project implements the operational half of the machine-learning lifecycle: MLflow for experiment tracking and a model registry, a containerized FastAPI inference service, batch scoring into the dashboard, and a monitoring job covering data quality, feature drift measured by Population Stability Index, predictive performance, scoring latency and model age. Those monitoring criteria drive a weekly Airflow retraining DAG that deliberately short-circuits unless a criterion actually fires. The entire stack runs under Docker Compose and is validated by 60 automated tests. Overall, the project demonstrates a complete MLOps pipeline — from raw data ingestion through model deployment to automated monitoring and retraining — rather than a model trained once in a notebook.

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude and respect to CMR Institute of Technology, Bengaluru for providing me a platform to pursue my studies and carry out this Mini Project.

It gives me immense pleasure to express my deep sense of gratitude to Dr. Sanjay Jain, Principal, CMRIT, Bengaluru, for his constant encouragement.

I would like to extend my sincere gratitude to Dr. Shyam P Joy, HOD, Department of Artificial Intelligence and Machine Learning, CMRIT, Bengaluru, who has been a constant support and encouragement throughout the course of this project.

I would like to thank my guide Dr Swastisudha Punyatoya, for the valuable guidance throughout the tenure of this Mini Project work.

I would also like to thank all the faculty members of the Artificial Intelligence and Machine Learning department who directly or indirectly encouraged me.

Finally, I thank my parents and friends for all the moral support they have given me during the completion of this work.

GROUP MEMBERS LIST

S. NO.	NAME OF STUDENT	USN
01	Rachit Joseph	1CR23CI041

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Certificate	2
2	Declaration	3
3	Abstract	4
4	Acknowledgement	5
5	Group Members List	6
6	Table of Contents	7
7	Introduction and Objectives	8
8	System Design and Working	11
9	Implementation	16
10	Results and Conclusion	20

1. INTRODUCTION AND OBJECTIVES

Retail and e-commerce businesses generate transaction records continuously, yet most of that data sits in operational systems that were never designed to answer analytical questions. Turning it into something a business can act on requires two distinct capabilities: a reliable data pipeline that consolidates and validates the records into an analytical model, and predictive models built on top of that model which are kept working after they are deployed. This project builds both, as a single integrated system, over a real public retail dataset.

This section presents the background motivation behind the project, the problem being addressed, the specific objectives of the system, and its overall scope. The focus lies on how data engineering and MLOps practices — orchestration, validation, experiment tracking, a model registry, containerized serving, monitoring and automated retraining — combine to produce a machine-learning system that remains trustworthy over time.

1.1. Background and Motivation

Customer retention is substantially cheaper than customer acquisition, which makes the ability to identify customers who are about to stop purchasing directly valuable. However, most academic machine-learning work stops at the point where a model reports a good accuracy score in a notebook. In practice that is the easy half. A model in production faces problems that training never exposes: the input data changes shape, upstream pipelines fail silently, the statistical distribution the model learned drifts away from the one it is now scoring, and nobody notices until the predictions have been wrong for weeks.

The discipline that addresses this is MLOps — applying software and operations engineering practice to the machine-learning lifecycle. Its central ideas are that data pipelines must be orchestrated and idempotent rather than run by hand, that every training run must be tracked and every model version registered so results are reproducible, that models must be served through a versioned interface, and above all that a deployed model must be continuously monitored and retrained on evidence rather than on a fixed schedule.

The motivation behind this project lies in building a system that demonstrates all of those practices end to end on a genuine dataset, rather than demonstrating a single model in isolation. A particular motivation is the retraining question: retraining a healthy model on a schedule is a common and harmful default, because it quietly replaces a working model with a worse one. This project therefore makes retraining conditional on measured evidence of degradation.

1.2. Problem Definition

The project addresses two linked problems. The first is a data-engineering problem: consolidating 25 monthly files of raw retail transactions — containing cancellations, missing customer identifiers, negative quantities and invalid prices — into a validated, query-efficient dimensional warehouse, repeatably and without duplication.

The second is a machine-learning problem: predicting, for each active customer at a given point in time, whether they will place no further order in the following 90 days. This is a binary

classification problem on tabular behavioural features, and its principal difficulty is not the choice of classifier but the correctness of the problem setup.

Key challenges include:

- Target leakage. Features must be computed strictly from data on or before each snapshot cutoff. A single feature that peeks past the cutoff inflates every metric and makes the model useless in production, while still looking excellent in evaluation.
- Temporal validity of the split. A random train/test split lets a customer's future inform their own prediction. The split must cut the data in time instead.
- Class imbalance and threshold choice. At a 90-day window roughly 58% of eligible customers churn, so accuracy is a weak metric and the decision threshold must be tuned deliberately.
- Defining the eligible population. A customer silent for two years has already churned rather than being at risk of churning, so including them would make the task trivially easy and operationally meaningless.
- Operational robustness. Pipelines must be idempotent, services must be reproducible across machines, and degradation must be detected automatically.

Addressing these challenges required treating the problem framing, the data pipeline and the operational tooling as being as important as the model itself.

1.3. Objectives

The objectives of the system are as follows:

- To design and implement an orchestrated, idempotent ETL pipeline that loads raw retail transactions through explicit validation into a PostgreSQL star schema with aggregate marts.
- To build an interactive analytics dashboard over the warehouse, covering revenue and order trends, product performance, customer segmentation, geography, retention, and data quality.
- To engineer leakage-free, point-in-time customer features and a well-defined inactivity-based churn label on month-end snapshots.
- To train and compare multiple candidate classifiers on a chronological split, select on validation performance, tune the decision threshold, and evaluate once on a strictly later held-out block.
- To track every experiment and register every model version in MLflow so that results are reproducible and auditable.
- To serve the selected model through a containerized REST API supporting both online and batch prediction.
- To implement monitoring of data quality, feature drift, predictive performance, scoring latency and model age, and to drive automated retraining from those criteria rather than from a schedule.

- To containerize the entire stack with Docker Compose and validate it with an automated test suite, so the system is reproducible on any machine.

Through these objectives the project demonstrates not only the modelling knowledge required to build a classifier, but the engineering practice required to operate one.

2. SYSTEM DESIGN AND WORKING

The system follows a layered architecture in which data moves through progressively more refined stages, and a machine-learning lifecycle is layered on top of the resulting warehouse. Each layer has a single responsibility and a defined contract with the next, so that a failure or a change is localised rather than propagating through the whole system.

This section describes the overall design and functionality, covering the pipeline topology, the dataset and its validation, the warehouse and feature design, the model and training configuration, and the serving and deployment workflow.

2.1. System Overview

The system is designed as an end-to-end pipeline orchestrated by two Apache Airflow DAGs. The first, retail_etl, moves transaction data from landing files into the warehouse. The second, churn_ml, operates the model lifecycle on top of it. All services run as containers under Docker Compose.

Workflow Summary:

1. Monthly landing CSVs are placed in the data directory by a one-time split script.
2. retail_etl task extract_landing_files ingests any not-yet-loaded file into raw.sales_raw, recording each load in a metadata table so repeat runs skip it.
3. Task validate_and_stage applies business rules, writing conforming rows to staging.sales_valid and non-conforming rows to staging.rejected_records with a reason.
4. Task transform_to_star_schema loads dimension and fact tables and refreshes the aggregate mart tables.
5. The Streamlit dashboard queries the warehouse and marts directly to render its analytical views.
6. ml.features builds point-in-time snapshot features and the churn label from the validated transactions.
7. ml.train fits the candidate models, selects one on the validation block, tunes the threshold, logs the run to MLflow and registers the winner.
8. The FastAPI service resolves the current model through the registry and serves online predictions; ml.score_churn produces batch scores for the dashboard.
9. ml.monitoring evaluates data quality, drift, performance, latency and model age, and emits a retraining recommendation that the churn_ml DAG acts on.

[paste screenshot here — Fig 1: architecture diagram from docs/architecture.md, or the Airflow DAG graph view]

Fig 1. Workflow Summary

This architecture separates concerns cleanly: the warehouse is the single source of truth for analytics, the feature layer is the single definition of what the model sees, and the registry is the single answer to the question of which model is current. Every consumer of the model —

the API, the batch scorer and the monitoring job — resolves it through the same registry module, so they can never disagree about which version is in use.

2.2. Dataset and Preprocessing

The project uses the public UCI Online Retail II dataset, which contains the transactions of a UK-based online retailer between December 2009 and December 2011. The source workbook is downloaded from the UCI Machine Learning Repository and split into 25 monthly landing CSV files by a one-time script. No final analytical table is edited by hand at any point; every table is produced by code from the source files.

Validation Rules:

Raw retail data is not clean, and the validation layer makes its rejections explicit and auditable rather than silently dropping rows. Each rejected row is written to staging.rejected_records together with the rule that rejected it:

- missing_customer_id — the row cannot be attributed to a customer, so it cannot support customer-level analysis or features.
- cancelled_order — invoice numbers prefixed with C denote cancellations, which are not sales.
- invalid_quantity — non-positive quantities, typically returns or adjustments.
- invalid_unit_price — non-positive prices, which would corrupt every revenue aggregate.

Validation Outcome:

Stage	Rows	Share
Raw rows ingested (raw.sales_raw)	1,067,371	100.0%
Passed validation (staging.sales_valid)	768,873	72.0%
Rejected with recorded reason	261,822	24.5%
Fact rows loaded (dw.fact_sales)	768,865	72.0%

Table 1. Validation and load outcome

The rejection rate of roughly one quarter of all rows is itself an analytical finding, and the dashboard surfaces it in a dedicated data-quality panel rather than hiding it. Idempotency is enforced at two levels: already-loaded files are skipped using the ingestion log, and the fact load deduplicates on the natural key, so re-running the DAG produces identical row counts.

Input Specifications:

- Source: UCI Online Retail II (https://archive.ics.uci.edu/dataset/502/online+retail+ii)
- Period covered: December 2009 – December 2011 (25 monthly files)
- Storage: PostgreSQL 16, schemas raw / staging / dw / mart / meta
- Warehouse model: star schema — dw.fact_sales with customer, product and date dimensions

2.3. Feature and Label Design

The machine-learning layer does not consume transactions directly. Instead it constructs customer-month snapshots: at each month-end cutoff, every eligible customer contributes one row of features summarising their behaviour up to that moment, together with a label describing what they did next.

Churn Label:

A customer is labelled churned at a cutoff if they place no order in the following 90 days. Only customers with at least one order in the preceding 365 days are eligible to be scored, because a customer who has already been silent for longer than a year has churned rather than being at risk of churning. This produced 62,607 labelled snapshots across 16 month-end cutoffs from May 2010 to August 2011, with an overall churn rate of approximately 58%.

Feature Set:

Twelve numeric features are computed, all strictly from data on or before the cutoff. They fall into three groups: classical RFM measures, behavioural breadth measures, and time-windowed trend measures.

Feature	Definition
recency_days	Days from the customer's last purchase to the cutoff
frequency	Number of distinct invoices placed up to the cutoff
monetary	Total revenue generated up to the cutoff
average_order_value	Monetary divided by frequency
tenure_days	Days from the customer's first purchase to the cutoff
distinct_products	Count of distinct stock codes purchased
avg_items_per_order	Mean quantity of items per invoice
avg_interpurchase_days	Mean gap in days between consecutive orders
orders_last_90d	Invoices placed in the 90 days before the cutoff
revenue_last_90d	Revenue generated in the 90 days before the cutoff
revenue_trend_ratio	Revenue in the last 90 days over revenue in the prior 90 days
distinct_active_months	Number of distinct calendar months with a purchase

Table 2. Feature set

Leakage prevention is treated as a correctness property and tested, not assumed. An automated test appends an entire extra year of future transactions to the input and asserts that not one feature value changes. That test is the strongest guarantee in the project that the reported metrics are achievable in production.

2.4. Model and Training Configuration

Three candidate classifiers were implemented as scikit-learn pipelines, so that preprocessing is bound to the estimator and cannot drift apart from it between training and serving. Each

pipeline carries its own preprocessing: the linear model standardises its inputs, while the tree-based models do not require scaling.

Candidate	Configuration
Logistic Regression	Standardised inputs, max_iter=1000, class_weight="balanced" — an interpretable linear baseline
Random Forest	n_estimators=300, max_depth=12, min_samples_leaf=20, class_weight="balanced"
Gradient-Boosted Trees	XGBoost: n_estimators=400, max_depth=5, learning_rate=0.05, subsample=0.9, colsample_bytree=0.9, reg_lambda=1.0, scale_pos_weight from the training class ratio

Table 3. Candidate models

Chronological Split:

The split cuts the snapshot cutoffs in time rather than sampling rows at random, so the evaluation answers the question actually being asked: having trained on the past, how well does the model predict the future? No cutoff is shared between blocks, so no customer's future can leak into their own training row.

Block	Snapshots	Rows	Cutoff range
Train	9	32,342	31 May 2010 – 31 Jan 2011
Validation	3	12,934	28 Feb 2011 – 30 Apr 2011
Test	4	17,331	31 May 2011 – 31 Aug 2011

Table 4. Chronological train / validation / test split

Selection and Threshold:

Candidates are compared on validation ROC-AUC, which measures ranking quality independently of any threshold. The decision threshold is then tuned separately, on the validation block only, to maximise F1; the selected threshold of 0.34 is well below the naive 0.5 because missing a churning customer is more costly than contacting a contented one unnecessarily. The test block is evaluated exactly once, after both the model and the threshold are fixed, so it remains a genuine estimate of unseen performance.

2.5. Serving and Deployment

The trained model is persisted as a joblib artifact and simultaneously registered as a version of the named model retail-churn-classifier in the MLflow registry. Every consumer resolves the model through a shared registry module, which prefers the registry and falls back to the local artifact, so a single code path decides which version is live.

Serving Interfaces:

- POST /predict — online scoring of a single customer feature vector, returning a churn probability and the thresholded decision.

- GET /model — the live version, threshold, feature list and training metrics, so a caller can verify what is actually serving.
- GET /health — liveness and whether a model is loaded.
- GET /metrics — request counts, failure counts and latency percentiles.
- GET /customers/{id}/churn — the latest batch score for one customer.
- POST /reload — picks up a newly trained artifact without rebuilding the container.

Deployment:

The whole stack is defined in a single Docker Compose file: PostgreSQL, the Airflow initialiser, scheduler and webserver, an MLflow tracking server backed by PostgreSQL, the FastAPI service and the Streamlit dashboard. Containerizing the stack is what makes the result reproducible: the same compose file brings the identical system up on any machine with Docker installed, which is the difference between a project that works and a project that works only on its author's laptop.

3. IMPLEMENTATION

The implementation phase turns the design above into running software. It covers the development environment, the ETL implementation, the training pipeline, the evaluation approach, and the monitoring and retraining logic. This section explains the practical setup and the engineering decisions made while building the system.

3.1. Development Environment

The system is implemented in Python 3.11+ and runs entirely in containers, with a host-side virtual environment available for development and for running the model pipeline without Docker.

Libraries and Tools Used:

- Apache Airflow (LocalExecutor) — orchestration of both DAGs.
- PostgreSQL 16 — warehouse storage and the MLflow backend store.
- pandas, NumPy — data handling, feature construction and transforms.
- scikit-learn — pipelines, the linear and random-forest candidates, and metrics.
- XGBoost — the gradient-boosted candidate.
- MLflow — experiment tracking and the model registry.
- FastAPI and Uvicorn — the inference service.
- Streamlit and Plotly — the analytics dashboard.
- SQLAlchemy — database access, through a shared engine helper.
- pytest — the automated test suite.
- Docker and Docker Compose — reproducible deployment of all services.

A notable implementation constraint was dependency resolution between these components. Airflow 2.10 pins SQLAlchemy below 2.0, while pandas 2.2 and above require SQLAlchemy 2.0 or newer. Where those constraints met, pandas silently fell back to treating a SQLAlchemy engine as a raw database connection and every database read and write failed. Resolving this required removing the project's own dependency on pandas 2.2 by writing the one version-sensitive call — the month-end frequency alias — so that it is valid in every pandas version. This kind of conflict is invisible in a notebook and only appears once the system is containerized, which is precisely the value of containerizing it.

3.2. ETL Implementation

The retail_etl DAG implements three sequential tasks: extract_landing_files, validate_and_stage and transform_to_star_schema. All three are idempotent, which was an explicit design requirement rather than an afterthought, because an ETL pipeline that cannot safely be re-run cannot safely be scheduled.

Implementation Details:

- Incremental extraction. Each landing file's load is recorded in a metadata ingestion log. On re-run, already-loaded files are skipped; a verification re-run reported skipped=25 and no change in row counts.
- Explicit validation. Rules are applied in a single pass, and every rejected row is written with its reason rather than discarded, so the rejection rate is auditable.
- Deduplicating load. The fact load deduplicates on the natural key, so a partially completed run can simply be re-run.
- Concurrency control. The DAG is configured with max_active_runs=1. Without it, two overlapping runs each passed the already-loaded check before either had written its marker, double-ingesting every file — a race condition that only appears under concurrency and produces silently doubled data.

The second DAG, churn_ml, runs weekly and implements the sequence check_model_health, should_retrain, retrain_model, score_customers and verify_after_retrain. The second task is a short-circuit: unless monitoring reports a firing criterion, the retraining and scoring tasks are skipped entirely.

3.3. Model Training Process

The training pipeline is a single reproducible command that reads transactions from PostgreSQL — or falls back to the landing files when the warehouse is unavailable — and produces a selected, registered, documented model.

Training Workflow:

10. Load validated transactions and build the month-end snapshot feature matrix with the churn label.
11. Split the snapshot cutoffs chronologically into 9 train, 3 validation and 4 test cutoffs.
12. Fit all three candidate pipelines on the training block, with class weighting derived from the training class ratio.
13. Score each candidate on the validation block and select the highest validation ROC-AUC.
14. Tune the decision threshold on the validation block to maximise F1.
15. Evaluate the selected model and threshold once on the held-out test block.
16. Log parameters, metrics, feature importances and artifacts to MLflow and register the model version.
17. Write the joblib artifact, the metrics JSON, the drift baseline and the model card.

A model card is generated automatically on every run, recording intended use, the label definition, the eligible population, the feature list, the split strategy, metrics, feature importances, monitoring thresholds, limitations, ethical considerations and the exact library versions used. Generating it as part of the run rather than writing it by hand means the documentation cannot fall out of date with the model.

3.4. Evaluation Metrics

Because the positive class is the majority at roughly 58%, accuracy is a weak and misleading metric here; a model predicting churn for everyone would score 58% while being useless. The evaluation therefore uses a set of complementary metrics:

- ROC-AUC — threshold-independent ranking quality; the primary model-selection metric.
- Precision — of the customers flagged as churning, the share who actually churn; this governs wasted retention spend.
- Recall — of the customers who actually churn, the share the model flags; this governs missed opportunity.
- F1 — the harmonic mean of precision and recall, used to tune the decision threshold.
- Average precision — summarises the precision-recall curve, which is more informative than ROC under imbalance.
- Brier score — measures how well-calibrated the predicted probabilities are, not merely how well they rank.
- Confusion matrix — exposes the actual counts behind the rates, including the false-positive volume.

3.5. Hyperparameters and Regularization

Hyperparameters were chosen to control overfitting on a dataset of moderate size, and are logged to MLflow on every run so that any result can be traced to the configuration that produced it.

Parameter	Value	Purpose
n_estimators	300	Number of trees; averages out variance
max_depth	12	Caps tree depth to limit overfitting
min_samples_leaf	20	Forces each leaf to generalise over 20+ samples
class_weight	balanced	Corrects for the class imbalance
Decision threshold	0.34	Tuned on validation to maximise F1
random_state	fixed	Makes every run reproducible

Table 5. Selected model hyperparameters

Regularization and Validity Techniques:

- Depth and leaf-size limits — restrict model capacity so the forest cannot memorise individual customers.
- Class weighting — prevents the majority class from dominating the fitted decision boundary.
- Chronological validation — the strongest guard against optimistic evaluation, since it forbids learning from the future.

- Threshold tuned on validation only — keeps the test block genuinely unseen.
- Leakage test — asserts automatically that no feature responds to post-cutoff data.

Together these measures produced a model whose validation and test performance agree closely — validation ROC-AUC 0.783 against test ROC-AUC 0.782 — which is itself the clearest evidence that the model generalises rather than having been overfitted or tuned against the test data.

4. RESULTS AND CONCLUSION

This section presents the outcomes of the system after a complete clean run of every component. It summarises the pipeline results, the model's measured performance, the monitoring outcome and the retraining decision, followed by a discussion of insights and future improvements.

4.1. Experimental Results

Pipeline Results:

The retail_etl DAG ran green from an empty database, ingesting 1,067,371 raw rows from 25 monthly landing files. Validation passed 768,873 rows and rejected 261,822 with recorded reasons, and 768,865 fact rows were loaded into the star schema. A second trigger changed no counts, confirming idempotency.

Model Selection:

All three candidates were trained and logged to MLflow. The random forest achieved the highest validation ROC-AUC and was selected and registered as retail-churn-classifier. The validation block (12,934 rows, 62.3% churn) gave ROC-AUC 0.783, precision 0.725, recall 0.944 and F1 0.820 at the tuned threshold of 0.34.

Held-Out Test Results:

The test block — four snapshot cutoffs from May to August 2011, 17,331 rows, strictly later than every training and validation cutoff — was evaluated once, after the model and threshold were fixed:

Metric	Test value	Reading
ROC-AUC	0.782	Good ranking quality; matches validation 0.783
Precision	0.677	About two thirds of flagged customers do churn
Recall	0.952	The model catches 95% of actual churners
F1	0.791	At the validation-tuned threshold of 0.34
Accuracy	0.709	Reported for completeness; weak under 58% imbalance
Average precision	0.800	Summarises the precision-recall curve
Brier score	0.184	Probabilities rank well but are not fully calibrated

Table 6. Held-out test performance (17,331 rows, threshold 0.34)

The confusion matrix on the test block was 9,573 true positives, 2,713 true negatives, 4,562 false positives and 483 false negatives. That shape is the intended consequence of the 0.34

threshold: the model is deliberately biased towards recall, accepting that some contented customers will be contacted in order to miss very few genuine churners.

[paste screenshot here — Fig 2: MLflow experiment run comparison showing the three candidates]

Fig 2. Candidate comparison in MLflow

[paste screenshot here — Fig 3: Airflow DAG graph view for retail_etl and churn_ml, all tasks green]

Fig 3. Airflow DAG runs

[paste screenshot here — Fig 4: Streamlit dashboard views, and a successful /predict response from the API]

Fig 4. Dashboard and API output

4.2. Model Performance

The close agreement between validation ROC-AUC 0.783 and test ROC-AUC 0.782 across four later snapshot cutoffs indicates that the model has learned behavioural patterns that persist over time rather than artefacts of a particular period. The leakage test passes, so these figures are achievable in production and not the product of a feature peeking past its cutoff.

Feature Importances:

The learned importances are consistent with established retail-analytics understanding, which is a useful sanity check on the feature engineering:

Feature	Importance	Interpretation
distinct_active_months	0.148	Breadth of engagement over time is the strongest signal
monetary	0.148	Total historical value
frequency	0.130	Number of orders placed
revenue_last_90d	0.090	Recent spending level
distinct_products	0.074	Breadth of catalogue purchased
avg_interpurchase_days	0.071	Habitual purchase rhythm
orders_last_90d	0.069	Recent order count
recency_days	0.065	Time since last purchase

Table 7. Top feature importances (random forest)

That habitual engagement breadth and monetary value dominate, rather than recency alone, is informative: the model is distinguishing customers whose buying habit has weakened from those who are simply between orders — which is exactly the distinction a retention campaign needs.

Serving Performance:

Batch scoring of 4,269 active customers completed with a mean batch latency of 113 milliseconds and a 95th percentile of 118 milliseconds, averaging 0.026 milliseconds per

customer, well inside the 250-millisecond monitoring budget. The containerized API served all endpoints correctly, and the model reloads without a rebuild.

4.3. Monitoring and Retraining Results

The monitoring job was run against the scored population and demonstrates the system working as intended — including correctly raising an alert.

- Data quality: pass. 4,269 rows scored, no duplicate customers, no nulls, no non-finite values and no domain violations across all twelve features.
- Performance: pass. On the last closed snapshot (4,334 rows) the model scored ROC-AUC 0.765, precision 0.599 and recall 0.959, all above their configured floors.
- Latency: pass. 95th-percentile batch latency 118 ms against a 250 ms budget.
- Model age: pass. Freshly trained, against a 30-day staleness limit.
- Drift: significant. Three features breached the PSI alert threshold of 0.25 — tenure_days at 0.528, recency_days at 0.322 and revenue_trend_ratio at 0.297 — while the other nine remained stable or moderate.

The resulting status was action_required with a retraining recommendation. This result is worth reading carefully, because the drift it detected is genuine and explicable rather than spurious. The scoring cutoff sits roughly three months after the last training cutoff, and the drifting features are exactly the time-anchored ones: tenure and recency necessarily shift as the population ages, and the revenue trend ratio moves with the retailer's November-December seasonal peak. The monitoring job therefore did precisely what it was built to do — it detected that the data the model is now scoring no longer resembles the data it learned from, and recommended a retrain on that evidence.

This is the mechanism that distinguishes the project from a model trained once. The weekly churn_ml DAG consumes this recommendation and retrains only when a criterion fires; had all criteria passed, the retraining and scoring tasks would have been skipped. The registry accordingly holds version 2 of retail-churn-classifier, produced by the DAG's own retrain, and a verification task re-ran monitoring against the new model afterwards.

[paste screenshot here — Fig 5: MLflow model registry showing the registered versions of retail-churn-classifier]

Fig 5. Model registry versions

4.4. Conclusion and Future Work

This project successfully implements a complete, operational MLOps pipeline over a real retail dataset. It ingests and validates over a million transaction rows into a dimensional warehouse, surfaces them through an analytics dashboard, trains and selects a churn classifier under a rigorous chronological evaluation, serves it through a containerized REST API, and monitors it in production with automated, evidence-driven retraining. The whole stack was verified end to end in Docker from a clean state, and is covered by 60 automated tests.

The most valuable lessons were operational rather than algorithmic. Choosing the classifier mattered far less than framing the problem correctly: the chronological split, the point-in-time feature construction and the automated leakage test are what make the reported 0.782 ROC-AUC a trustworthy number rather than an optimistic one. Equally instructive was bringing the stack up in Docker, which surfaced seven distinct defects — a shell script with Windows line endings that prevented database initialisation, two separate dependency conflicts, a character-encoding failure that silently disabled all experiment logging, a race condition that double-ingested data under concurrent runs, and a missing serialisation dependency that left the API quietly serving a degraded model. Most of those failed silently while still reporting success, which is the strongest argument for the end-to-end verification and monitoring the project implements.

Future Enhancements:

- Probability calibration, so that scores can be multiplied by customer value to produce an expected-loss figure suitable for budgeting retention spend.
- Setting the decision threshold from campaign economics — the cost of an outreach against the margin of a retained customer — rather than from the F1 optimum.
- Adding explainability, such as SHAP values exposed through the API, so each flagged customer arrives with a reason a campaign manager can act on.
- A feature store to guarantee that the training and serving feature definitions can never diverge.
- Champion-challenger deployment, so a retrained model must out-perform the incumbent on live traffic before it is promoted.
- A CI/CD pipeline that runs the test suite and rebuilds the images automatically on every commit.
- Enriching the feature set with marketing-contact and web-session data, which would let the model describe causes of churn rather than only the behaviour preceding it.

In conclusion, this project represents a complete machine-learning lifecycle — from data ingestion and validation through feature engineering, model selection and containerized deployment to automated monitoring and conditional retraining — underscoring that a production machine-learning system is an engineering artefact to be operated, not a model to be trained once.